Objektumok modellezése és példányosítása

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Objektum különböző nézőpontokból

Modellezési szempontból

- Az objektum a megoldandó problémának egy adott részéért felelős, önálló egyedként azonosítható része.
- Az objektum egységbe zárja a felelősségi köréhez tartozó adatokat, és egyben elrejti őket, hogy azokat csak a metódusai révén olvassuk, változtassuk.
- Az objektumnak van életciklusa : objektum létrejöttével kezdődik, és megszűnésével fejeződik be.

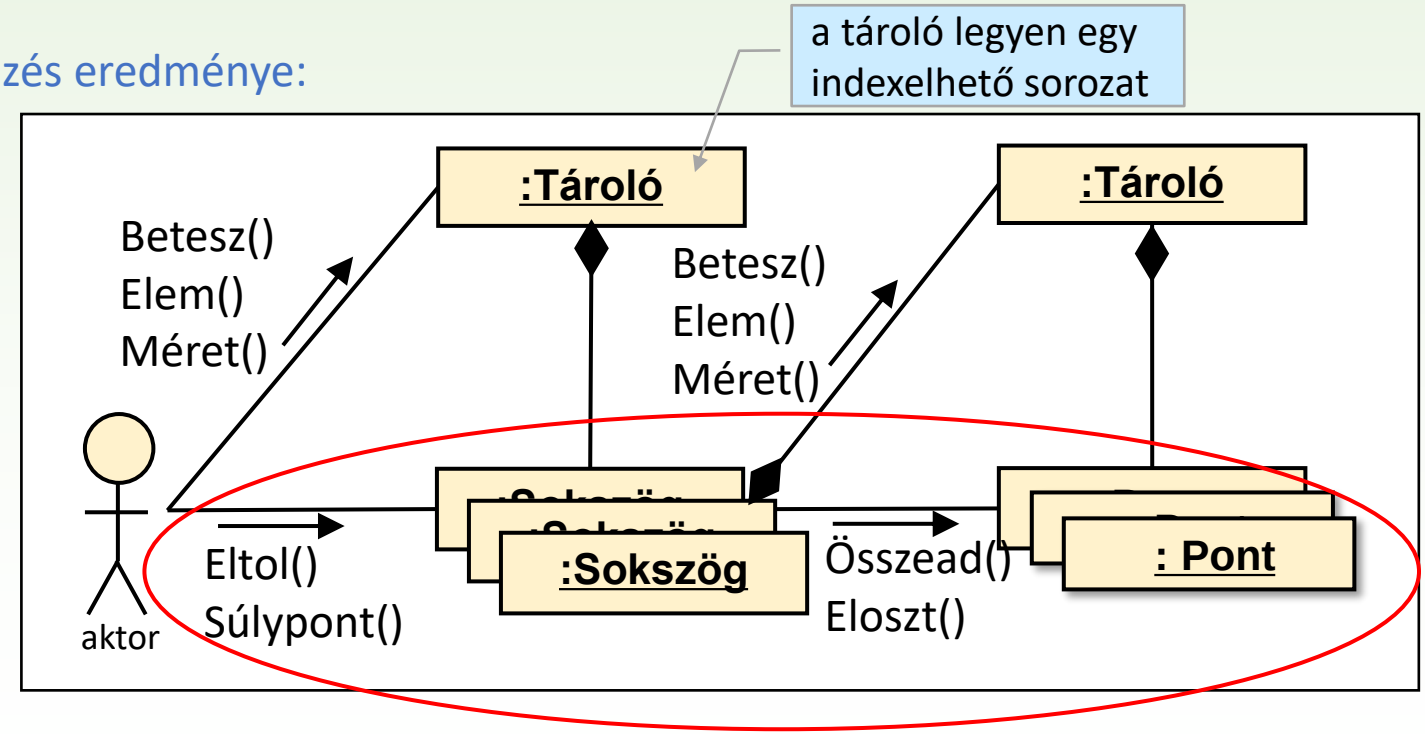
Programnyelvi megközelítés

- Az objektum egy olyan memória-foglalás, ahol az objektumhoz tartozó adatokat tároljuk.
- Az objektum adattagjainak és metódusainak) láthatósági köre szabályozható, de az objektum metódusai mindig elérik azokat.
- Egy objektumnak a konstruktora foglal memóriát (példányosítás), és a destruktora törli őt.

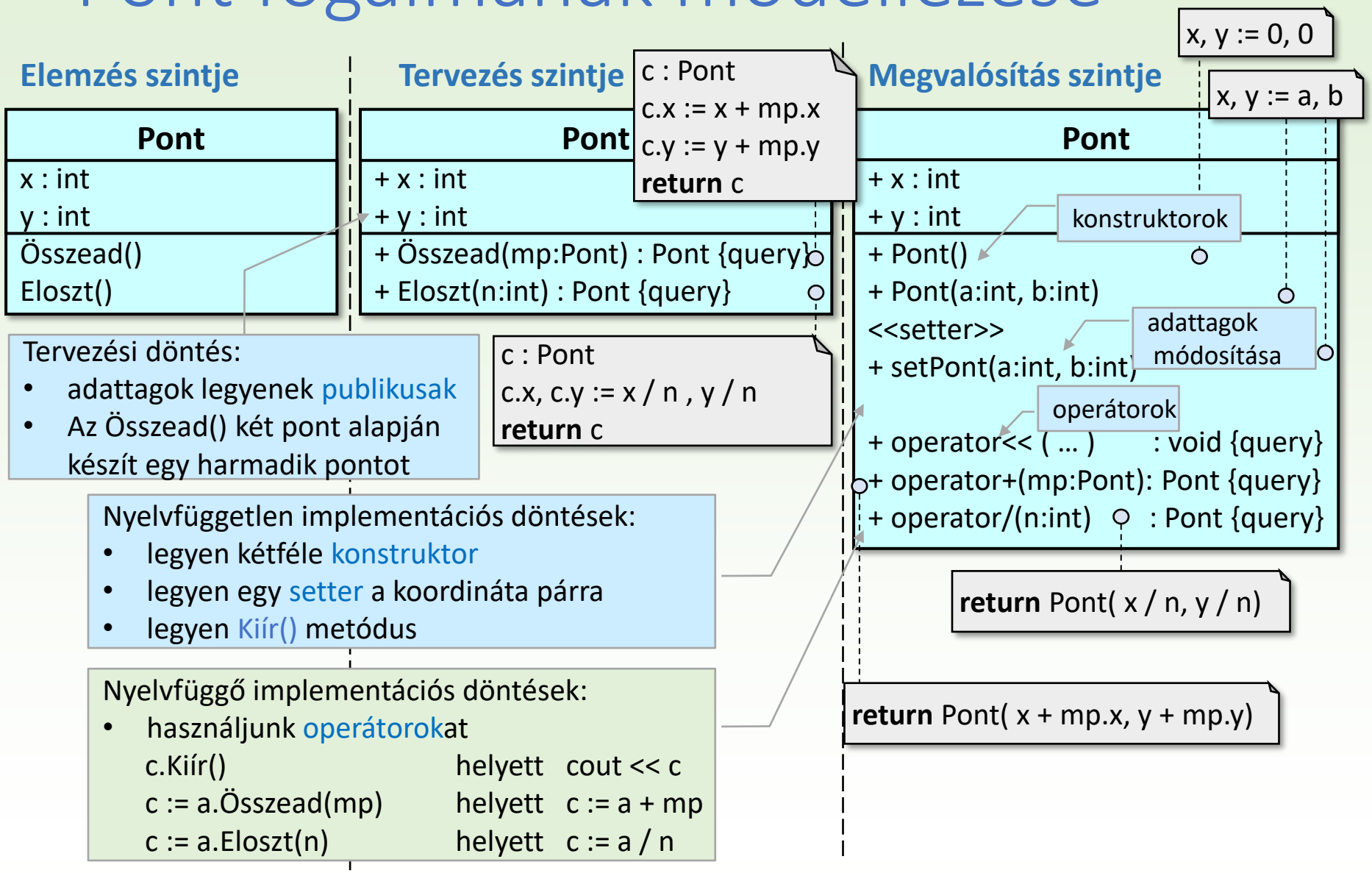
Feladat

Töltsünk fel egy tárolót különféle sokszögekkel, és mindegyiket toljuk el ugyanazon irányba és mértékkel, majd számoljuk ki az így nyert sokszögek súlypontjait. A csúcspontok és súlypontok koordinátái, sőt az eltolást leíró helyvektor végpontjának koordinátái is legyenek egész számok.

Elemzés eredménye:



Pont fogalmának modellezése



Pont osztály C++ kódja

Pont
+ x : int + y : int
+ Pont() + Pont(a:int, b:int) <<setter>> + setPont(a:int, b:int) + operator<< () : void {query} + operator+(mp:Pont): Pont {query} + operator/(n:int) : Pont {query}

```
class Point
{
public:
        int _x, _y;
public:
        Point(int x = 0, int y = 0): _x(x), _y(y) { }

        void setPoint(int x, int y) { _x = x; _y = y; }

        Point operator+(const Point &mp) const
                { return Point(_x + mp._x, _y + mp._y); }

        Point operator/(int n) const
                { return Point(_x / n, _y / n ); }
};
```

paraméterváltozók default értéke:
Point a;
Point b(3);
Point c(-4, 8);

konstans referencia

query

operátor felüldefiniálás:
a p + q a p objektumra hívja meg az operator+(q)-t, ahol a q-ra az mp hivatkozik, és egy pontot ad majd vissza.

A kiíró operátor nem a Point osztály metódusa, hisz cout<<p formában kell a cout-ra meghívni, nem pedig p<<... alakkal a p pontra.
Ezért külső operátorként vezetjük be.

```
std::ostream& operator<<(std::ostream &o, const Point &p)
{
        o << "(" << p._x << "," << p._y << ")";
        return o;
}
```

Objektum default pointere

```
class Point
{
public:
        int _x, _y;
public:
        Point(int x=0, int y=0):_x(x),_y(y) {}

        void setPoint(int x, int y) { _x = x; _y = y; }
        void setPoint(int x, int y) { this->_x = x; this->_y = y; }
        …

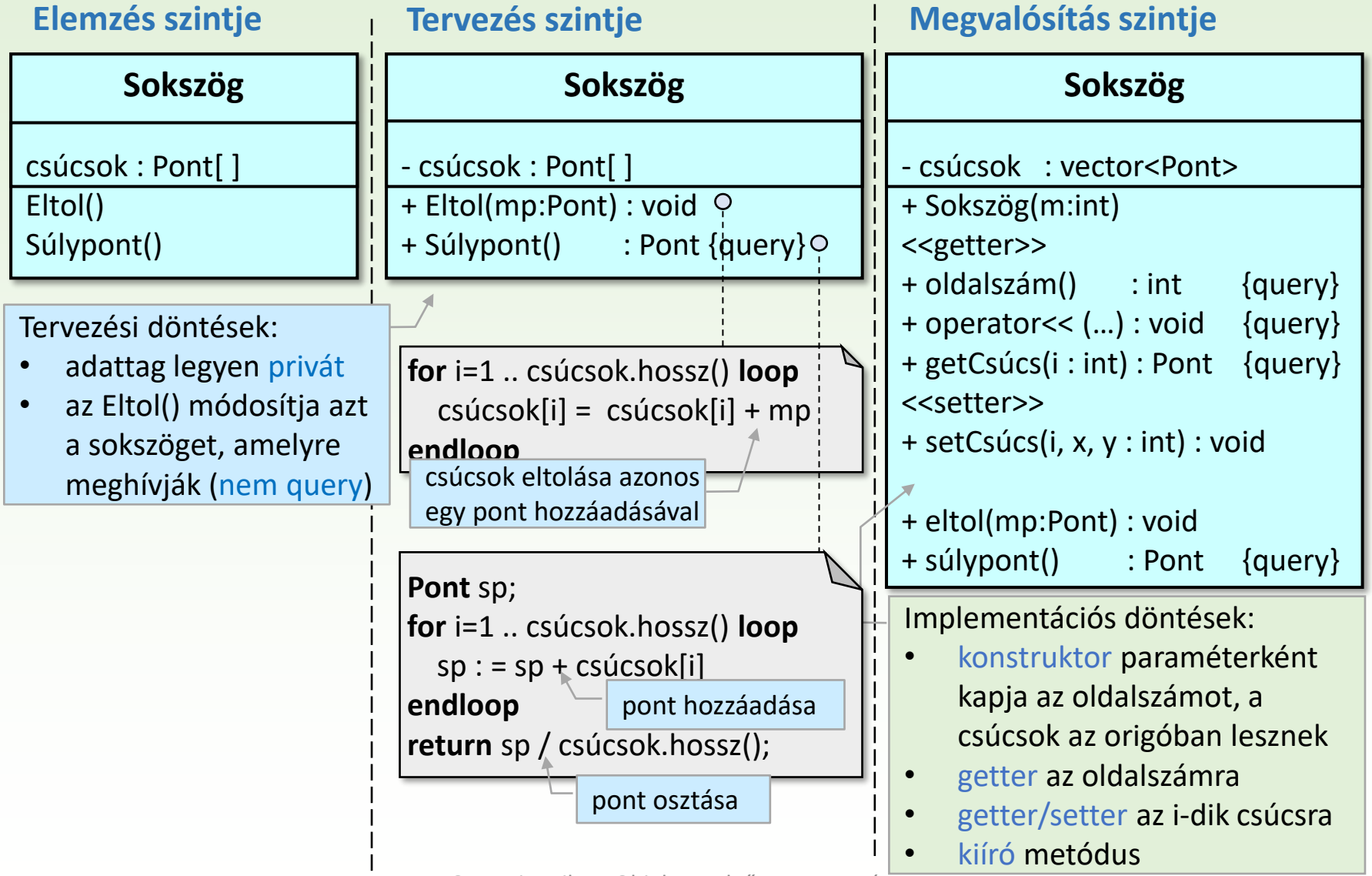
};
```

A **this** egy alapértelmezett módon létező pointerváltozó, amely azon objektumnak a memóriacímét tartalmazza (arra mutat), amelyik objektumra a metódust meghívták: Pl. a p.setPoint(3, -2) hívás esetén a this a p címe.

3

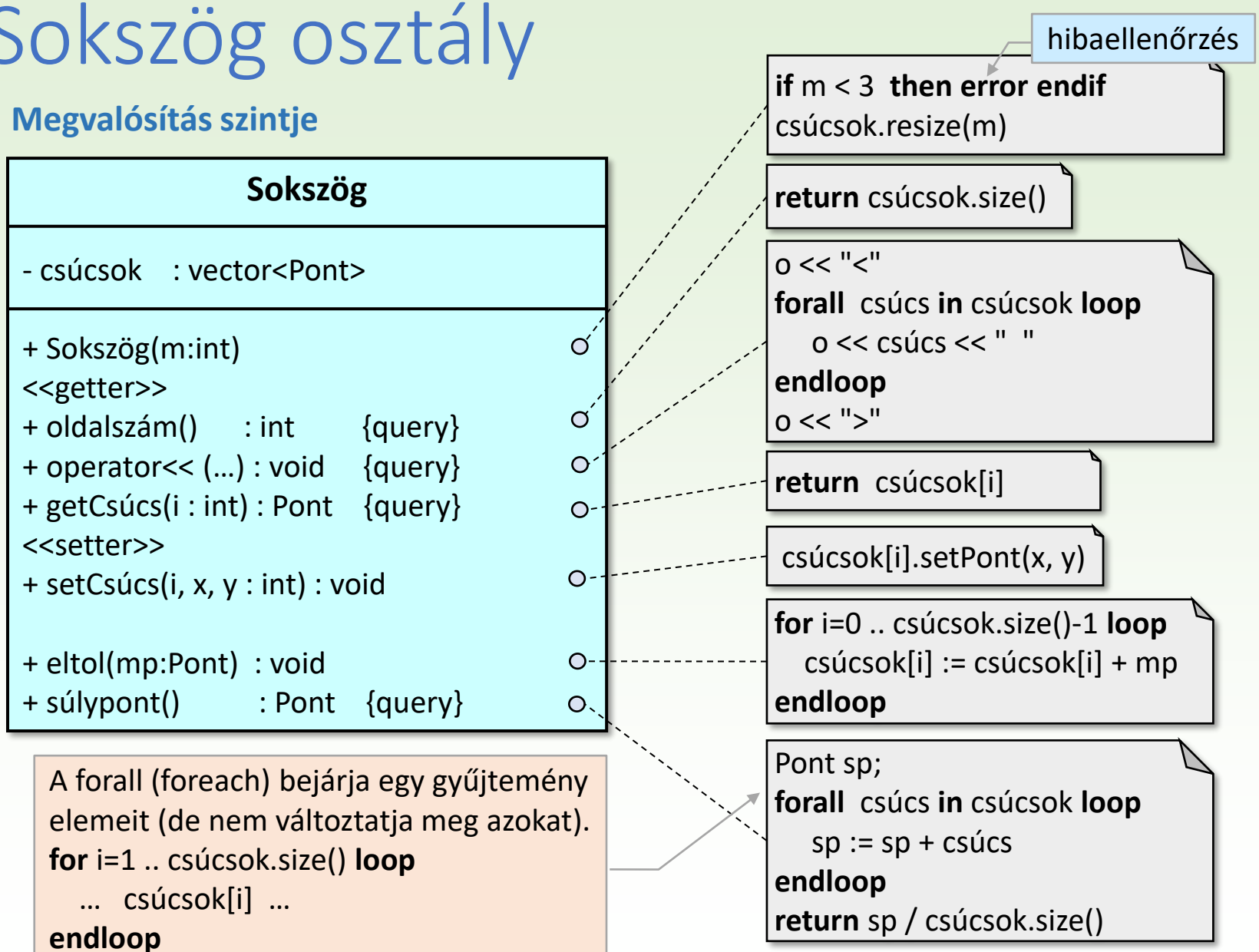
Az objektum orientált nyelvek további ismérve a nyílt rekurzió:
az objektum mindig látja saját magát, eléri saját műveleteit és adattagjait.

Sokszög fogalmának modellezése



Sokszög osztály

Megvalósítás szintje



Sokszög osztály C++ kódja

```
class Polygon{
public:
        class FewVerticesException : std::exception {};
        class IllegalIndexException : std::exception {};

        Polygon(int m) : _vertices(m)  {
                if (m < 3) throw new FewVerticesException;
        }
        // getters
        int  sides() const { return (int)_vertices.size(); }
        friend std::ostream& operator<<(std::ostream &o, const Polygon &p);
        Point getNode(int i) const { return _vertices[i]; }
        // setter
        void setNode(int i, int x, int y) {
                if (i < 0 || i>=(int)_vertices.size()) throw new IllegalIndexException;
                _vertices[i].setPoint(x,y);
        }
        // methods
        void  move(const Point &mp);
        Point center() const;
private:
        std::vector<Point> _vertices;
};
```

kivételek definiálása

kivétel eldobása

olyan külső operátor, amelyik hozzáfér a privát tagokhoz

kivétel eldobása

Sokszög metódusainak C++ kódja

```
void Polygon::move(const Point &mp)
{
	for(unsigned int i=0; i<_vertices.size(); ++i) {
		_vertices[i] = _vertices[i] + mp;
	}
}

Point Polygon::center() const
{
	Point center;
	for(const Point& vertex : _vertices) { center = center + vertex; }
	return center / sides();
}

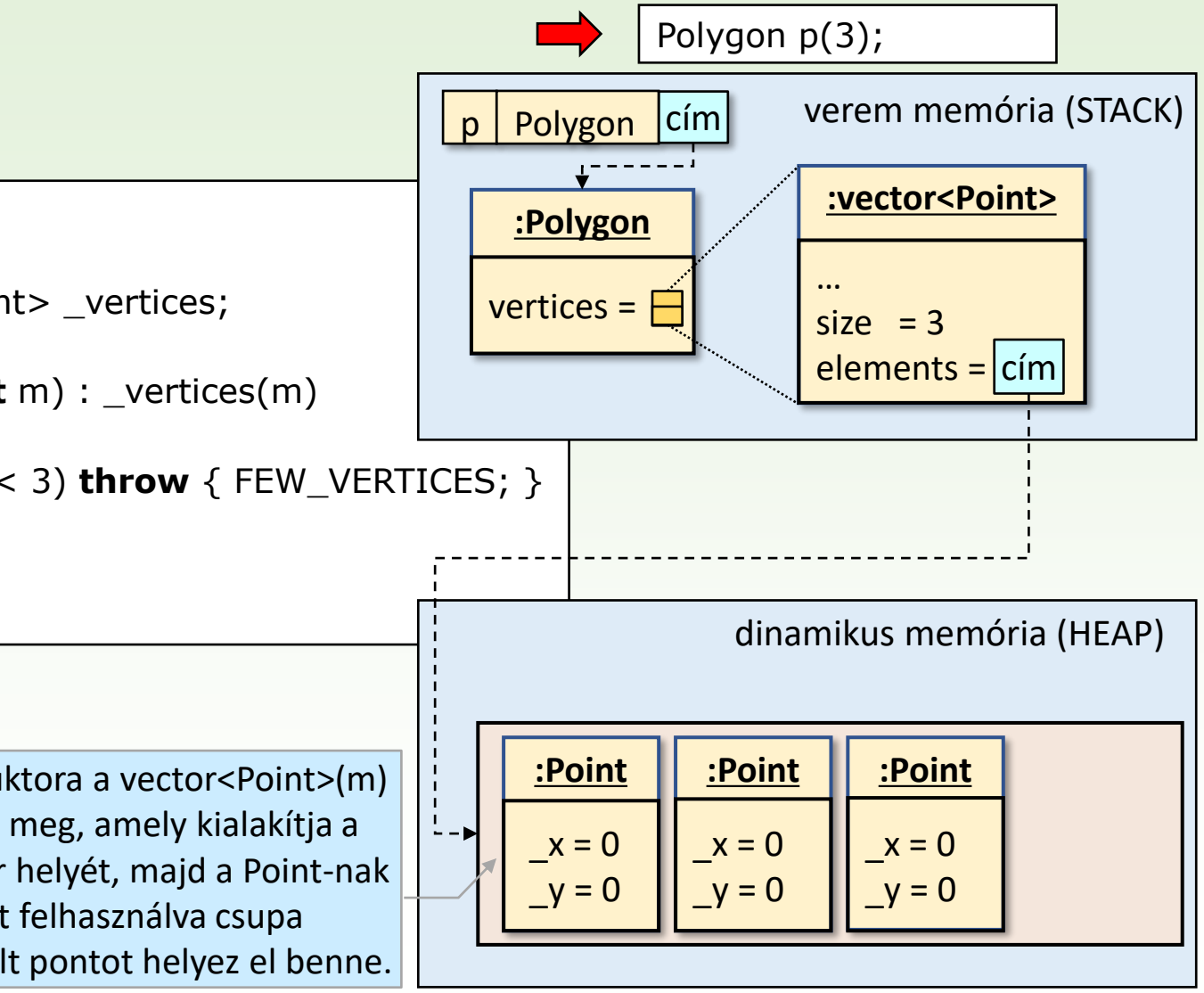
std::ostream& operator<<(std::ostream &o, const Polygon &p)
{
	o << "<";
	for( const Point& vertex : p._vertices ) { o << vertex; }
	o << ">";
	return o;
}
```

a **forall** (foreach) ciklus C++ kódja

a friend kapcsolat miatt hivatkozhat a privát adattagra

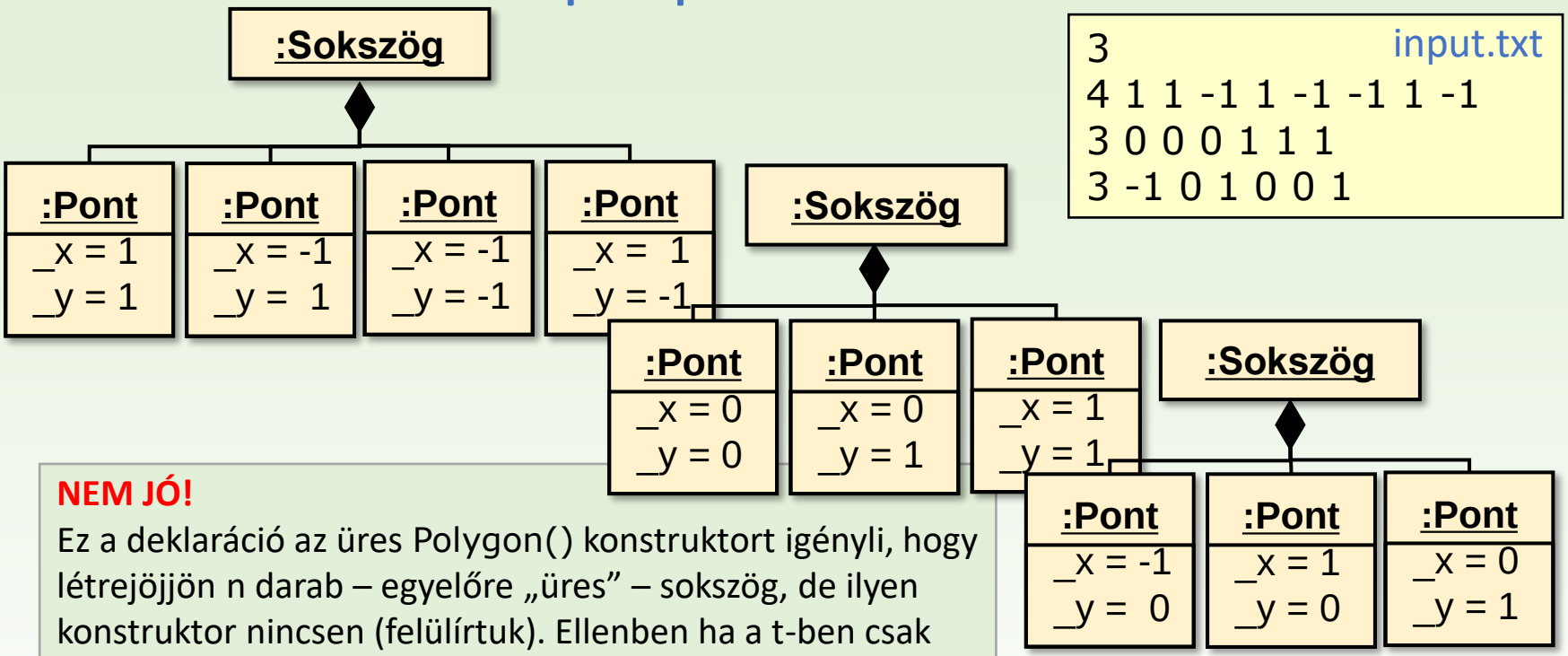
Sokszög példányosítása

```
class Polygon {
private:
      vector<Point> _vertices;
public:
      Polygon(int m) : _vertices(m)
      {
            if (m < 3) throw { FEW_VERTICES; }
      }
      …
};
```



A Polygon konstruktora a vector<Point>(m) konstruktort hívja meg, amely kialakítja a heap-ben a vector helyét, majd a Point-nak üres konstruktorát felhasználva csupa origóba pozícionált pontot helyez el benne.

A feladat felpopulálása



input.txt

```
3
4 1 1 -1 1 -1 -1 1 -1
3 0 0 0 1 1 1
3 -1 0 1 0 0 1
```

NEM JÓ!

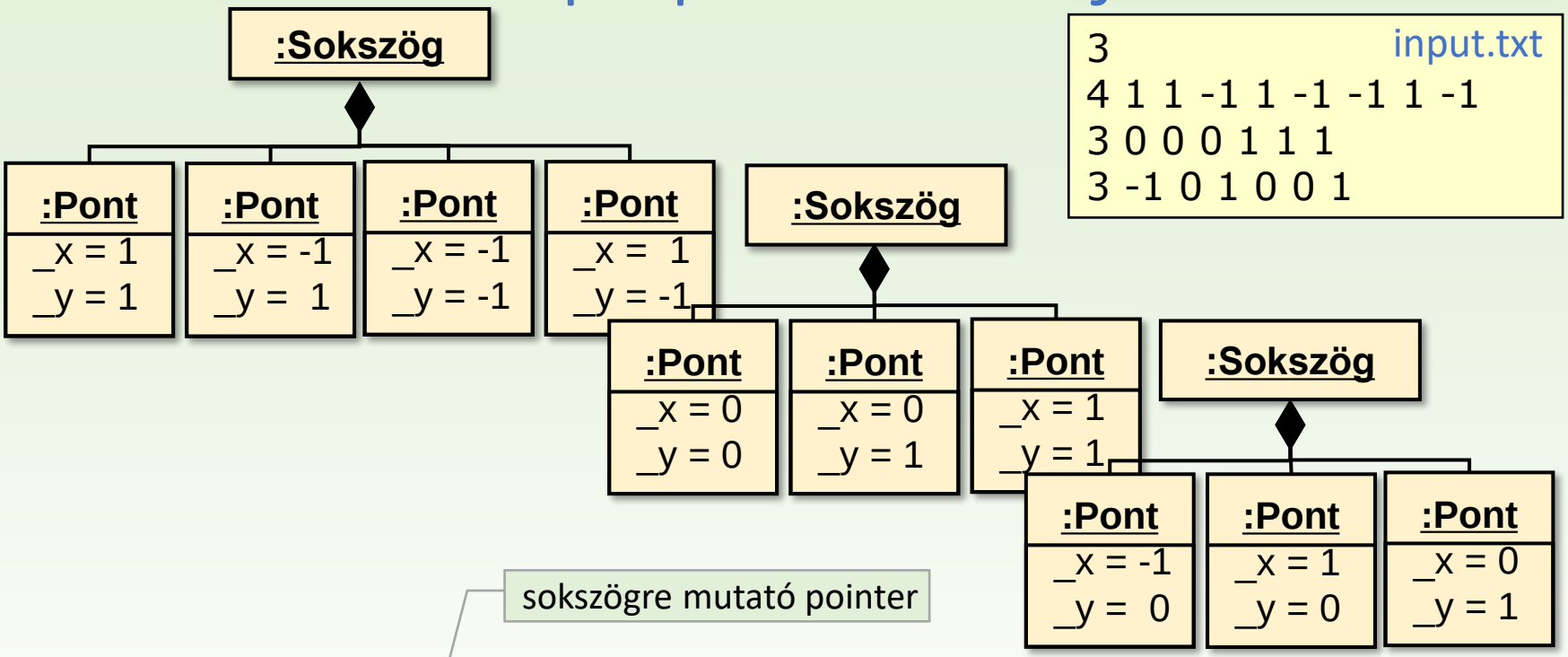
Ez a deklaráció az üres Polygon() konstruktort igényli, hogy létrejöjjön n darab – egyelőre „üres” – sokszög, de ilyen konstruktor nincsen (felülírtuk). Ellenben ha a t-ben csak sokszögekre mutató címeket, azaz pointereket tárolnánk, akkor elég lenne a sokszögeket majd később létrehozzuk.

```
ifstream inp("input.txt");
...
int n; inp >> n;
vector<Polygon> t(n);

for( int i=0; i<n; ++i ) { t[i].set(inp); }
```

Kell egy olyan metódus, ami a szöveges állomány soron következő sora alapján beállítja a sokszög oldalainak számát, és a sokszög csúcsainak koordinátáit.

A feladat felpopulálása újra



input.txt

```
3
4 1 1 -1 1 -1 -1 1 -1
3 0 0 0 1 1 1
3 -1 0 1 0 0 1
```

sokszögre mutató pointer

Kell egy metódus, ami a szöveges állomány soron következő sora alapján létrehoz egy új sokszöget, és visszaadja annak címét. Ez nem lehet a létrehozandó objektum metódusa, mert azt egy még nem létező objektumra nem lehetne meghívni. Legyen osztályszintű metódus, hogy láthassa az objektum rejtett tagjait.

```
ifstream inp("input.txt");
…
int n; inp >> n;
vector<Polygon*> t(n);

for( int i=0; i<n; ++i ) { t[i] = create(inp); }
```

Osztályszintű gyártó függvény

```
Polygon* Polygon::create(ifstream &inp)
{
	Polygon *p;
	try
	{
		int sides;
		inp >> sides;
		p = new Polygon(sides);
		for(int i=0; i < sides; ++i) {
			int x, y; inp >> x >> y;
			p->setNode(i, x, y);	// p->_vertices[i].setNode(x, y);
		}
	}
	catch(Polygon::FewVerticesException e)
	{
		cout << "cannot create the
	}
	return p;
}
```

input.txt

```
4 1 1 -1 1 -1 -1 1 -1
```

létrehozza a sokszöget, de kivételt dob, ha sides<3

beállítja a sokszög csúcsainak koordinátáit

```
class Polygon {
public:
	enum Errors { … };
	Polygon(int n);
	…
	static Polygon* create(std::ifstream &inp);
private:
	std::vector<Point> _vertices;
};
```

osztályszintű

osztályszintű metódus hívása

```
t[i] = Polygon::create(inp);
```

Főprogram

Eltárolt sokszögek eltolása ugyanazon irányban és mértékkel, majd súlypontjaiknak kiszámolása.

A = (t : Sokszög* , mp : Pont , cout : Pont*)

Ef = ($t = t_0 \wedge mp = mp_0$)

Uf = ($mp = mp_0 \wedge t = \bigoplus_{i=1}^{n} < eltol(t_0[i], mp) > \wedge cout = \bigoplus_{i=1}^{n} < súlypont(t[i]) >$)

sorozatok összefűzésének jele

Két összegzés (összefűzés):

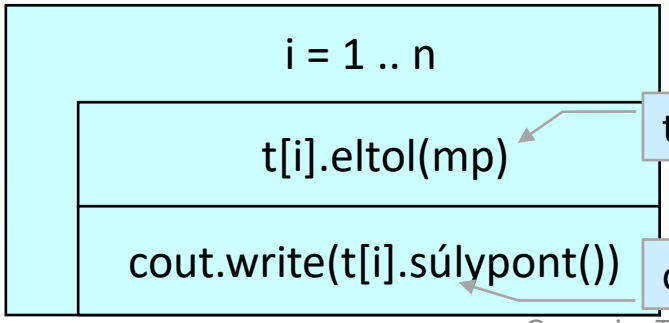
i∈[m..n]	~	i∈[1 .. n]	i∈[1 .. n]
f(i)	~	<eltol(t_0[i], mp)>	<súlypont(t[i])>
s	~	t	cout
H,+,0	~	Sokszög*, ⊕, <>	Pont*, ⊕, <>

a két összegzés közös ciklusba vonható össze

az összefűzés helyett kiírás alkalmazunk

Nem kell ide a t, cout := <>, <>

az összefűzés helyett elég a már létező t elemeit egyenként megváltoztatni: t[i] := eltol(t[i], mp)



t[i] := eltol(t[i], mp)

cout := cout ⊕ súlypont(t[i])

```
for( Polygon *p : t ) {
    p->move(mp); cout << p << endl;
    cout << p->center() << endl;
}
```

Típus-orientált procedurális kód

```
#include "polygon.h"
#include "point.h"
using namespace std;

int main()
{
        cout << "file name: "; string fn; cin>> fn;
        ifstream inp(fn);
        if(inp.fail()) { cout << "File open error\n"; return 1;}
        int n; inp >> n;
        vector<Polygon*> t(n);
        for ( int i=0; i<n; ++i ) t[i] = Polygon::create(inp);
        int x, y; inp >> x >> y;
        Point mp(x, y);
        for ( Polygon* p : t ) {
                p->move(mp);
                cout << *p << endl;
                cout << p->center() << endl;
        }
        for ( Polygon* p : t ) delete p;
        return 0;
}
```

#include <iostream>
#include <fstream>
#include <vector>

populálás

számolás

törlés

megszünteti a dinamikus memóriában tett helyfoglalásainkat

Objektum-orientált kód

```
int main(){
        Application a;
        a.run();
        return 0;
}
```

```
class Application{
public:
        Application();
        void run();
        ~Application();
private:
        std::vector<Polygon*> t;
        Point mp;
};
```

```
Application::Application(){
        cout << "file name: "; string fn; cin>> fn;
        ifstream inp(fn);
        if(inp.fail()) { cout << "File open error\n"; exit(1);}
        int n; inp >> n;
        t.resize(n);
        for( int i=0; i<n; ++i) t[i] = Polygon::create(inp);
        int x, y; inp >> x >> y;
        mp.setNode(x, y);
}
```

populálás

#include <cstdlib>

```
void Application::run(){
        for ( Polygon* p : t ) {
                p->move(mp); cout << *p << endl;
                cout << p->center() << endl;
        }
}
```

számolás

A destruktor akkor hívódik meg, amikor az objektum megszűnik.

```
Application::~Application(){
        for ( Polygon* p : t ) delete p;
}
```

törlés

Menüvezérelt objektum-orientált kód

```
int main()
{
	Menu a;
	a.run();
	return 0;
}
```

```
void Menu::run()
{
	int v = 0;
	do{
		menuWrite();
		cin >> v;  // ellenőrzés!
		switch(v){
			case 1: case1(); break;
			case 2: case2(); break;
			case 3: case3(); break;
			case 4: case4(); break;
		}
	}while(v != 0);
}
```

```
class Menu{
public:
	Menu(){s = nullptr;}
	void run();
	~Menu(){ if(s!=nullptr) delete s;}
private:
	Polygon* s;
	void menuWrite();
	void case1();
	void case2();
	void case3();
	void case4();
};
```

egy sokszöget létrehozó, kiíró, eltoló, súlypontját kiszámoló metódusok

```
void Menu::menuWrite(){
	cout << "0 – exit\n";
	cout << "1 – create\n";
	cout << "2 – write\n";
	cout << "3 – move\n";
	cout << "4 – center\n";
}
```

Menüpontok

input1.txt

```
4 1 1 -1 1 -1 -1 -1 1
```

input2.txt

```
3 0 0 -1 0 0 -1
```

```
void Menu::case1(){ // create
      if(s!=nullptr) delete s;
      cout << "file name: "; string fn; cin>> fn;
      ifstream inp(fn);
      if(inp.fail()) { cout << "File open error\n"; return;}
      s = Polygon::create(inp);
}
```

```
void Menu::case2(){  // write
      if(s==nullptr) { cout << "There is no polygon!\n"; return;}
      cout << *s << endl;
}
```

```
void Menu::case3(){  // move
      if(s==nullptr) { cout << "There is no polygon!\n"; return;}
       ... //  x, y beolvasása
      Point mp(x, y);
      s->move(mp);
      cout << *s << endl;
}
```

```
void Menu::case4(){  // center
      if(s==nullptr) { cout << "There is no polygon!\n"; return;}
       cout << s->center() << endl;
}
```